So if `HttpClient` can already use connection pooling, is it worth using `IHttpClientFactory`? In most cases, I would say yes. You must manually configure connection pooling with `SocketsHttpHandler`, and `IHttpClientFactory` has additional features such as named clients and typed clients. In any situations where you're using dependency injection (DI), which is every ASP.NET Core app and most .NET 7 apps, I recommend using `IHttpClientFactory` to take advantage of these benefits.

Nevertheless, if you're working in a non-DI scenario and can't use `IHttpClientFactory`, be sure to enable the `SocketsHttpHandler` connection pooling as described in this post by Steve Gordon, titled "HttpClient connection pooling in .NET Core": http://mng.bz/E27q.

Managing the socket problem is one big advantage of using `IHttpClientFactory` over `HttpClient`, but it's not the only benefit. You can also use `IHttpClientFactory` to clean up the client configuration, as you'll see in the next section.

33.2.2 Configuring named clients at registration time

In this section you'll learn how to use the Named Client pattern with `IHttpClientFactory`. This pattern encapsulates the logic for calling a third-party API in a single location, making it easier to use the `HttpClient` in your consuming code.

NOTE `IHttpClientFactory` uses the same `HttpClient` type you're familiar with if you're coming from .NET Framework. The big difference is that `IHttpClientFactory` solves the DNS and socket exhaustion problem by managing the underlying message handlers.

Using `IHttpClientFactory` solves the technical problems I described in section 33.1, but the code in listing 33.3 is still pretty messy in